

QA Automation – Testes de API

Manual técnico — para desenvolvedores

Sumário

1. Antes de tudo: para que serve (e para que não serve)
2. Acesso e pré-requisitos
3. Como a ferramenta chama a sua API
4. O fluxo em 4 etapas
5. Referência: o formato de um caso de teste
6. Como o resultado é classificado
7. Evidências e o que chega num Bug
8. Confrontando com o seu código — receitas
9. Limitações conhecidas
10. Dúvidas frequentes e solução de problemas

1. Antes de tudo: para que serve (e para que não serve)

O módulo **Testes de API** do QA TestGen faz requisições HTTP reais contra uma API publicada (por exemplo, o ambiente de Homologação), confere cada resposta contra regras declaradas e guarda a evidência de cada chamada (requisição, resposta e resultado). Ele foi feito para o trabalho de QA. Este manual existe para que você, desenvolvedor, consiga usar a mesma ferramenta — ou só entender o que ela reportou — e comparar com o código que você escreveu.

O que ele não pretende ser

Não substitui os testes do time (unitários, de integração, de contrato) nem a revisão de código. Quem conhece a regra de negócio e a implementação é você. A ferramenta olha a API de fora, como caixa-preta, do jeito que o front-end e o QA a enxergam. É bem provável que parte do que ela aponta já seja conhecida pelo time — nesse caso, ela serve apenas como registro e evidência.

Quando costuma agregar

- **O ambiente publicado não é o ambiente local.** Configuração, variáveis de ambiente, modo de depuração, WAF, CORS, perfis de usuário e dados do ambiente podem fazer a mesma rota responder diferente do que responde na sua máquina. A ferramenta mostra o que o ambiente publicado devolve de fato.
- **Contrato visto de fora.** Nome real dos campos (snake_case × camelCase), status de validação (400 × 422), formato das mensagens de erro e headers expostos ao navegador — detalhes que o front consome e que às vezes divergem da especificação escrita.
- **Reproduzir exatamente o que o QA viu.** Cada apontamento vem com a requisição completa (com segredos mascarados) e a resposta bruta. Dá para repetir a mesma chamada, no mesmo ambiente, sem depender de descrição em texto.
- **Conferir uma correção.** Depois de um deploy, a mesma bateria roda de novo em segundos e mostra se o comportamento mudou.
- **Regressão entre entregas.** A definição da bateria (.json) pode ser guardada e executada de novo a cada deploy.

Quando provavelmente não precisa

- A regra já está coberta por testes automatizados do time e não depende de configuração do ambiente.
- O comportamento não aparece na resposta HTTP (efeitos em fila, e-mail, banco de dados) — a ferramenta só vê a resposta.
- Carga, concorrência e desempenho — as chamadas são sequenciais e o tempo é medido do lado de quem chama.

Como ler o que a ferramenta aponta

Toda classificação da análise é uma **hipótese com evidência**, não um veredito. As regras são heurísticas (descritas na seção 6) e erram para o lado conservador: quando não dá para saber qual comportamento é o certo, o item vai para "a confirmar com o PO", e não para "bug". Se algo reportado não fizer sentido diante do código, é bem possível que a bateria de testes esteja presumindo algo errado — a própria ferramenta tem uma categoria para isso ("problema da bateria"), e o seu retorno ajuda a corrigi-la.

💡 *Recebeu um Bug aberto por esta ferramenta? As seções 7 (o que chega num Bug) e 8.1 (reproduzir a chamada) são as mais úteis — não é preciso usar o app para verificar.*

2. Acesso e pré-requisitos

A ferramenta fica dentro do QA TestGen, no menu lateral, em  **Testes de API**.

- **Acesso:** usuário no QA TestGen com a permissão `testes_api`, liberada pelo administrador do app.
- **Base URL** do ambiente que você quer testar (ex.: `https://api.exemplo.com.br`). As rotas dos casos são relativas a ela, pela variável `{{base_url}}`.
- **Usuário(s) de teste** de cada perfil que as rotas exigem. As senhas são digitadas na hora, ficam só na memória da sessão e nunca vão para a IA nem para as evidências.
- **Opcional:** uma collection do Postman (v2.1), um arquivo OpenAPI/Swagger ou a lista de rotas da API, para importar casos ou o catálogo de rotas reais.
- **Opcional:** acesso ao Azure DevOps (PAT), só para gerar casos a partir de Work Items ou abrir Bugs.

Modo de execução

Uma configuração global decide de onde saem as requisições. Ela vale para todos os usuários e só o administrador altera (Administração →  Configurações).

Modo	De onde sai a requisição	O que isso exige da API
Navegador do usuário (padrão)	Do navegador de quem usa o app, via <code>fetch</code>	CORS liberado para a origem do app; <code>preflight</code> aceitando <code>Authorization</code> e <code>Content-Type</code> ; headers de resposta que você queira verificar listados em <code>Access-Control-Expose-Headers</code>
Servidor do app	Do servidor onde o QA TestGen roda, em Python (<code>requests</code>)	O IP do servidor precisa passar pelo WAF/firewall do ambiente

 O padrão é o navegador porque WAFs como o do CloudFront costumam bloquear IPs de provedores de nuvem (onde o app está hospedado), mas aceitam o IP de quem está testando — o mesmo IP que o seu Postman usa.

3. Como a ferramenta chama a sua API

Nos dois modos o resultado é avaliado pelo mesmo código (Python); o que muda é quem envia a requisição. Detalhes que costumam importar ao investigar do lado do servidor:

Aspecto	Modo navegador	Modo servidor
Como identificar nos seus logs	Header Origin com o endereço do QA TestGen; User-Agent do navegador	User-Agent: QA-TestGen-ApiTests/1.0
Corpo da requisição	Enviado exatamente como está no caso; não é enviado em GET/HEAD	Enviado exatamente como está no caso, em qualquer método (UTF-8)
Content-Type	Só o que estiver nos headers do caso — nada é adicionado	Idem
Cookies	Não são enviados nem guardados	Só os que a própria API definir (Set-Cookie) durante a mesma execução — são reenviados nas chamadas seguintes
Redirecionamentos	Seguidos	Seguidos
Certificado TLS	Validado pelo navegador	Validado (certificados do sistema)
Timeout	Configurável na etapa 1 (5 a 300 s, padrão 30 s)	Idem
Headers de resposta visíveis	Só os liberados pelo CORS (os básicos + Access-Control-Expose-Headers)	Todos
Headers que não dá para enviar	Os proibidos pelo navegador (ex.: User-Agent, Host, Cookie)	Sem restrição
Ordem	Sequencial, na ordem da bateria	Idem

Chamadas feitas antes da bateria (sondas)

Duas funções chamam a API por conta própria, **sem credencial**, para descobrir se uma rota existe:  **Reconhecer a API** (antes de gerar casos com IA) e  **Verificar rotas dos casos**. Cada sonda usa o método real da rota, troca parâmetros de caminho ({id}, {{variavel}}) por 1 e envia corpo {} em POST, PUT e PATCH.

 Uma rota de escrita que não exija autenticação recebe a sonda de verdade (por exemplo, DELETE /recursos/1). Em rotas protegidas a sonda recebe 401 e para por aí — que é o esperado.

A resposta da sonda é lida assim: 404 com mensagem do tipo "route ... could not be found" (ou 404 sem corpo JSON) significa que a rota não existe; qualquer outra resposta (401, 403, 405, 422...) significa que existe.

4. O fluxo em 4 etapas

Etapa 1 — Definição

Identificação (nome, ambiente, Base URL, timeout), origem dos casos, variáveis e, opcionalmente, um contexto que vai para o relatório. Os casos podem vir de:

-  **Gerar com IA** — a especificação vem de Work Items do Azure DevOps (Descrição e Critérios de Aceite) ou de texto/documentos colados. A IA recebe a especificação, o catálogo de rotas reais e o que o app já aprendeu sobre o formato da API. Nunca recebe senhas nem tokens: só declara as variáveis.
-  **Collection do Postman** — v2.1, com environment opcional. Os `pm.test` mais comuns viram asserções (seção 5.6).
-  **Definição salva** — o `.json` exportado na etapa 4 (formato na seção 5). É o jeito mais direto de um dev escrever os próprios casos.
-  **Criar manualmente** — caso a caso, na tela.

Catálogo de rotas reais

Para a IA não inventar rotas, o app guarda, por host, a lista das rotas que existem. Ela pode ser importada do JavaScript do front-end publicado na Base URL (o app lê os bundles e extrai as chamadas HTTP), de um OpenAPI/Swagger, de uma collection do Postman ou de texto colado. Casos com rota fora do catálogo ficam desabilitados com aviso, e rotas que respondem durante as execuções entram no catálogo sozinhas.

Variáveis

- `{{nome}}` em URL, headers, body e nos valores das asserções é substituído antes do envio. `{{base_url}}` é sempre o campo Base URL.
- **Secretas** (senhas, tokens) são digitadas em campo de senha, ficam só na sessão e saem mascaradas de toda evidência.
- **Extração:** um caso pode guardar um valor da resposta numa variável (ex.: `data.token` → `auth_token`) para os casos seguintes — por isso a ordem dos casos importa.
- **Variável sem valor não trava a execução:** só os casos que a usam saem como Bloqueado, sem enviar a requisição.
- **Preenchimento automático:** tokens de outros perfis ganham um caso de login próprio; dados de teste negativo recebem valores fixos (token inválido, e-mail inexistente, senha errada, ID 999999999).

Etapa 2 — Execução

Roda os casos habilitados, em sequência, e mostra o resultado de cada asserção. O significado de cada resultado está na seção 6.1.

Etapa 3 — Análise e Bugs

Classifica o que não passou (seção 6), sugere correções de formato na própria bateria e permite criar rascunhos de Bug (ver, alterar, excluir) e enviá-los ao Azure DevOps — sempre com confirmação e com o link de cada item criado. O relatório completo (PDF/MD) pode ser baixado antes do envio.

Etapa 4 — Evidências

Relatório em Markdown e PDF, pacote `.zip` com uma pasta por caso e a definição `.json` da bateria (seção 7).

5. Referência: o formato de um caso de teste

A definição exportada na etapa 4 — e importada em "Definição salva" — é um JSON neste formato. Senhas saem em branco na exportação e são pedidas de novo ao importar.

```
{
  "formato": "qa_testgen.api_tests.v1",
  "projeto": "Minha API",
  "ambiente": "Homologação",
  "base_url": "https://api.exemplo.com.br",
  "timeout": 30,
  "variaveis": [
    { "nome": "email", "valor": "qa@exemplo.com", "secreto": false },
    { "nome": "senha", "valor": "", "secreto": true }
  ],
  "contexto": "",
  "observacoes": "",
  "casos": [
    {
      "id": "login",
      "nome": "Login com credenciais válidas (200)",
      "metodo": "POST",
      "url": "{{base_url}}/api/v1/auth/login",
      "headers": { "Content-Type": "application/json", "Accept": "application/json" },
      "body": "{ \"email\": \"{{email}}\", \"password\": \"{{senha}}\" }",
      "assercoes": [
        { "tipo": "status", "valor": "200", "descricao": "Login aceito" },
        { "tipo": "json_not_empty", "alvo": "data.token" }
      ],
      "extrair": [ { "nome": "auth_token", "caminho": "data.token" } ],
      "habilitado": true,
      "descricao": "Obtém o token usado pelos próximos casos"
    },
    {
      "id": "perfil",
      "nome": "Perfil do usuário logado (200)",
      "metodo": "GET",
      "url": "{{base_url}}/api/v1/me",
      "headers": { "Accept": "application/json", "Authorization": "Bearer {{auth_token}}" },
      "body": "",
      "assercoes": [
        { "tipo": "status", "valor": "200" },
        { "tipo": "json_type", "alvo": "data.id", "valor": "string" },
        { "tipo": "response_time_max", "valor": "1500" }
      ],
      "extrair": [],
      "habilitado": true
    }
  ]
}
```

5.1 Campos de um caso

Campo	Tipo	Observação
id	texto	Identificador único na bateria (vazio = o app gera um).
nome	texto	Aparece no relatório e vira o título do Caso de Teste no Azure DevOps.
metodo	texto	GET, POST, PUT, PATCH, DELETE, HEAD ou OPTIONS.
url	texto	Em geral {{base_url}}/caminho; aceita variáveis e query string.
headers	objeto	Nome → valor; aceita variáveis. Nada é adicionado automaticamente, nem Content-Type.
body	texto	Enviado exatamente como escrito — JSON precisa estar serializado como texto.
assercoes	lista	Regras da resposta (5.2). Caso sem nenhuma asserção é sempre reprovado.
extrair	lista	{ "nome", "caminho" }: guarda um valor da resposta JSON numa variável (5.5).
habilitado	booleano	false = não roda (aparece como "Não Executado").
descricao	texto	Opcional; vai para o relatório e para a Matriz de Cobertura.

5.2 Tipos de asserção

tipo	alvo	valor	Passa quando
status	—	código HTTP	o status recebido é igual
json_exists	caminho	—	o caminho existe na resposta
json_absent	caminho	—	o caminho não existe

tipo	alvo	valor	Passa quando
json_equals	caminho	valor	o valor é igual (regras em 5.3)
json_not_empty	caminho	—	o valor não é "", null, [] nem {}
json_type	caminho	string, number, boolean, array, object ou null	o tipo JSON bate
json_contains	caminho	trecho	o texto do valor contém o trecho (valor não-texto é comparado na forma JSON)
json_equals_var	caminho	nome da variável	o valor é igual ao guardado na variável (regras em 5.3)
header_contains	header	trecho	o header contém o trecho, sem diferenciar maiúsculas (nome também)
body_contains	—	trecho	o corpo bruto contém o trecho (diferencia maiúsculas)
body_not_contains	—	trecho	o corpo bruto não contém o trecho
response_time_max	—	ms	o tempo total, medido por quem chama, é até o valor

5.3 Regras de comparação (json_equals e json_equals_var)

- **Número:** comparado como número (1 e 1.0 são iguais).
- **Texto:** igual exatamente, ou igual ignorando maiúsculas e espaços nas pontas.
- **Booleano:** true/false, sem diferenciar maiúsculas.
- **null:** o esperado pode ser null, none ou vazio.
- **Objeto ou lista:** comparado com o JSON serializado, de chaves ordenadas.
- Variáveis {{x}} no alvo e no valor são substituídas antes da comparação.

5.4 Caminhos JSON aceitos

O executor aceita um subconjunto de JSONPath, igual nos dois modos de execução:

Sintaxe	Exemplo	Significado
campo aninhado	data.user.email	navega pelos objetos
índice	data.items[0] · data.items[-1]	primeiro · último item
todos os itens	data.items[*].id	o id de cada item
todos os valores	data.*	cada valor do objeto
filtro	data.items[?(@.status=='ativo')].id	itens que atendem; aceita ==, != e [?(@.campo)] (campo verdadeiro)
tamanho	data.items.length	quantidade de itens (ou de caracteres de um texto)
chave especial	data['campo com espaço']	chave com espaço ou símbolo
raiz	\$ · \$.data	o \$ inicial é opcional

Quando o caminho devolve vários valores ([*] ou filtro), basta um deles atender à regra — exceto em json_type, em que todos precisam ser do tipo. Busca recursiva (..) não é suportada. Caminho com sintaxe fora desta lista aparece como "sintaxe de caminho não suportada", nunca como "campo ausente".

5.5 Extração de variáveis

- Só de respostas JSON.
- Texto é guardado como está; número, objeto ou lista são guardados como JSON.
- Se o caminho devolver vários valores, fica o primeiro.
- Se não encontrar, a variável mantém o valor anterior; se estava vazia, os casos que dependem dela saem como Bloqueado.

5.6 Importação do Postman

Os scripts do Postman são JavaScript, e o executor não roda JavaScript. O importador reconhece, por padrão de texto, estes formatos dentro de pm.test:

No script do Postman	Vira
pm.response.to.have.status(200) · pm.expect(pm.response.code).to.eql(200)	status
pm.expect(obj).to.have.property('campo') · .to.not.have.property(...)	json_exists · json_absent
pm.expect(valor).to.eql(x) · .to.equal(x)	json_equals
pm.expect(valor).to.be.a('string')	json_type
pm.expect(valor)...not.empty	json_not_empty

No script do Postman	Vira
<code>pm.environment.set('nome', valor)</code> (também <code>collectionVariables</code> , <code>variables</code> , <code>globals</code>)	extração
comparação com valor lido por <code>pm.environment.get(...)</code>	<code>json_equals_var</code>

Também são importados: `auth Bearer` definida no request (vira header `Authorization`) e `body raw`, `urlencoded`, `form-data` (só campos de texto) e `GraphQL`. Não são importados: scripts de `pre-request`, `auth` definida na `collection` ou na pasta, `upload` de arquivos e qualquer lógica `JavaScript` fora desses padrões — cada caso com algo não convertido recebe um aviso na tela.

6. Como o resultado é classificado

6.1 Resultado de cada caso (etapa 2)

Resultado	Quando
Aprovado	Todas as asserções passaram.
Reprovado	Alguma asserção falhou, ou o caso não tem nenhuma asserção.
Erro	A requisição não teve resposta (rede, CORS, timeout, TLS), ou a API respondeu que a rota não existe (404 "route ... could not be found", ou 404 sem corpo JSON).
Bloqueado	O caso usa uma variável vazia — a requisição não é enviada, e o motivo diz qual variável e qual caso deveria produzi-la.
Não Executado	Caso desabilitado.

💡 Um 404 de "registro não encontrado" com corpo JSON (id inexistente, por exemplo) é resposta normal da API e é avaliado pelas asserções. Só o 404 de rota é tratado como Erro.

6.2 Categorias da análise (etapa 3)

Categoria	Significa	É bug?
🐞 Possível bug da API	Comportamento que merece Bug (6.3).	Provavelmente — confirmar
👤 Usuário sem o perfil exigido	403 com mensagem de papel/perfil exigido.	Não
📋 A confirmar com o PO	A API faz X, o teste esperava Y, e o card não decide.	Depende da regra
🔋 Problema da bateria	Token esquecido, campo com nome diferente do que a API valida, caminho presumido, sintaxe não suportada, rota inexistente.	Não
⌚ Tempo acima do limite	<code>response_time_max</code> estourado.	Só se houver requisito de tempo
🔊 Sem resposta	Rede, CORS ou timeout.	Repetir antes de concluir
⏸ Bloqueado	Dependência não resolvida, com a causa-raiz.	Não se aplica

Regra que evita culpar a API por engano: um status ou uma mensagem só são tratados como **requisito** se o texto do card (Work Item) citar aquele status ou aquela mensagem. Sem isso, a divergência vai para "a confirmar com o PO". O esperado nunca é ajustado automaticamente para bater com o obtido.

6.3 O que conta como possível bug

Achado	Regra de detecção	Gravidade sugerida
Erro interno	Status 5xx (502/503/504 são tratados como possível instabilidade de infraestrutura).	Alta
Detalhe interno exposto	Corpo com erro de SQL (SQLSTATE, ORA-), stack trace, caminho de arquivo do servidor, mensagem interna do framework (ex.: "No query results for model"), nome de classe, modelo ou exceção.	Média (SQL, stack trace, caminho) ou Baixa
Acesso sem credencial aceito	Resposta 2xx num caso que só aceita erro e que espera 401/403 ou testa "sem token" / "token inválido".	Crítica
Validação ausente	Resposta 2xx num caso que só aceita status 4xx/5xx.	Média
Campo sensível na resposta	Resposta 2xx com valor em password, password hash, senha, secret, client_secret, api_key, remember_token ou private_key.	Alta
Lentidão	Resposta acima de 3 s.	Baixa

Para cada achado, a análise diz se é bug mesmo, como confirmar, o que fazer, com quem falar e traz um texto pronto para a conversa. O app também aprende o formato real da API (campos validados, caminhos da resposta, mensagens por status) e sugere correções **só de formato** na bateria — nunca de status esperado nem de texto de mensagem.

7. Evidências e o que chega num Bug

7.1 Arquivos da etapa 4

- **RELATORIO.md e RELATORIO.pdf** — resumo, resultado por caso, análise, correções aplicadas e Bugs (rascunhos e enviados).
- **Pacote .zip** — uma pasta por caso (NN_nome-do-caso/) com 1_request.txt, 2_response.txt, 3_resultado.txt e os prints anexados.
- **Definição .json** — a bateria inteira, para executar de novo (seção 5).

7.2 Mascaramento

Antes de gerar qualquer arquivo, o app troca por *****MASCARADO*****:

- os headers **Authorization** (mantém o prefixo "Bearer"), **X-API-Key**, **Api-Key**, **Cookie** e **Set-Cookie**;
- os valores dos campos JSON **password**, **senha**, **token**, **access_token**, **refresh_token**, **secret**, **api_key**, **apikey** e **authorization**;
- qualquer token **Bearer** e qualquer **JWT** no texto;
- o valor de toda variável secreta e dos tokens obtidos durante a execução.

🔔 Por isso, para repetir a chamada de um Bug você precisa gerar o seu próprio token — o da evidência nunca vem em claro.

7.3 O que um Bug aberto pela ferramenta contém

- **Título** no padrão [API][HML] descrição do achado.
- **Descrição:** onde (Base URL, ambiente, bateria), os casos envolvidos (número no relatório da execução e nome), a evidência, a análise e a sugestão.
- **Passos de reprodução:** a autenticação usada (qual rota e qual campo do token) e a requisição completa, mascarada.
- **Resultado esperado × obtido.**
- **Anexos:** request/response de cada caso (.txt, mascarados) e, se quem enviou marcou, o RELATORIO.pdf.
- **Vínculos:** Related ao Work Item testado e Tested By aos Test Cases da bateria, quando existirem.
- **Severidade e prioridade** sugeridas pela gravidade — sempre revisadas por quem envia.

O app não edita nem exclui itens no Azure DevOps depois de criados; qualquer ajuste é feito lá.

8. Confrontando com o seu código — receitas

8.1 Reproduzir uma chamada fora do app

Pegue o `1_request.txt` do caso (no .zip ou anexado ao Bug). Ele tem método, URL final, headers e corpo. Troque o token mascarado por um seu e rode, por exemplo:

```
curl -i -X POST "https://api.exemplo.com.br/api/v1/recursos" \
  -H "Accept: application/json" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer SEU_TOKEN" \
  --data '{"campo": "valor"}'
```

Se a resposta vier diferente da registrada, a diferença costuma estar no usuário (perfil, empresa), nos dados do ambiente ou num deploy feito depois da execução — o horário da execução está no relatório.

8.2 Escrever seus próprios casos

Monte um .json no formato da seção 5 com o contrato que você implementou (status, campos, mensagens), importe em Etapa 1 → Origem → Definição salva, preencha as senhas e execute. Serve para registrar o comportamento esperado de uma rota nova e repetir a cada deploy.

8.3 Rodar a mesma bateria contra outro ambiente

Na etapa 1, troque a Base URL (todas as rotas usam `{{base_url}}`) e execute de novo. Para apontar para a sua máquina: no modo navegador a chamada sai do seu navegador, então o endereço local precisa estar acessível a ele e aceitar CORS da origem do app (alguns navegadores pedem permissões extras quando uma página publicada chama um endereço local); no modo servidor, o servidor do app precisa alcançar o endereço — o que em geral não acontece com `localhost`.

8.4 De um apontamento para o ponto do código

O app mostrou	Onde costuma estar a explicação
🚫 "Rota inexistente"	Registro de rotas, prefixo (ex.: <code>/api/v1</code>), método HTTP da rota, rota ainda não publicada no ambiente.
📋 Campo com nome diferente (o 422 lista outro campo)	Validação da requisição: nomes esperados (snake_case × camelCase).
📋 Status diferente do esperado (ex.: 422 × 400)	Status padrão da validação no framework × o que o card pede.
📋 Mensagem diferente	Chave de tradução × texto; idioma/locale do ambiente.
👤 403 de perfil	Middleware ou policy de papel; vínculo do usuário de teste (empresa, perfil).
⚙️ Detalhe interno exposto	Tratamento de exceções e modo de depuração do ambiente.
🔥 5xx	Log da aplicação no horário da execução (que está no relatório).
📋 Caminho presumido / campo ausente	Serialização da resposta (resource, DTO): nome e aninhamento dos campos.
💡 Sem resposta no modo navegador	Configuração de CORS: origens, headers permitidos, preflight.
Header esperado vazio no modo navegador	<code>Access-Control-Expose-Headers</code> .

8.5 Conferir uma correção

Depois do deploy, execute de novo a mesma definição (etapa 2): a análise da etapa 3 reflete a nova resposta na hora. O Bug no Azure DevOps pode ser acompanhado pelo botão  **Verificar no Azure** (estado, coluna e responsável).

9. Limitações conhecidas

- Não envia arquivos (multipart com upload); form-data do Postman vira campos de texto.
- Não executa JavaScript (scripts do Postman, pre-request).
- JSONPath é um subconjunto (seção 5.4).
- Execução sequencial: sem paralelismo, carga ou medição de concorrência. O tempo inclui a rede entre quem chama e a API.
- Asserções de campo exigem resposta JSON; para outros formatos, use `body_contains` e `header_contains`.
- Extração só de respostas JSON.
- No modo navegador: cookies não são enviados, headers proibidos não podem ser definidos, só os headers de resposta liberados pelo CORS ficam visíveis, e GET/HEAD não levam corpo.
- A classificação da análise é heurística e conservadora — serve para priorizar, não para decidir.
- Casos gerados por IA podem presumir detalhes. O catálogo de rotas e o aprendizado do formato reduzem isso, mas não eliminam — por isso existe a categoria "problema da bateria".

10. Dúvidas frequentes e solução de problemas

Mensagem	Causa provável	O que fazer
"Failed to fetch" / "Se a API não permitir CORS..."	O navegador bloqueou a chamada (CORS) ou a API não está acessível.	Conferir o CORS para a origem do app, ou pedir ao administrador o modo Servidor.
"tempo limite de N s excedido"	A resposta demorou mais que o timeout.	Aumentar o timeout na etapa 1, ou investigar a rota.
"resposta não é JSON válido"	O corpo não é JSON (HTML de erro, texto, arquivo).	Ver o <code>2_response.txt</code> ; usar <code>body_contains</code> ou <code>header_contains</code> .
"sintaxe de caminho não suportada"	Caminho fora da seção 5.4.	Reescrever o caminho.
"Bloqueado: a variável 'x' está vazia..."	A extração do caso anterior falhou, ou a variável não foi preenchida.	Corrigir o caso que deveria produzir o valor, ou preencher a variável.
"Variáveis não definidas: ..."	{{nome}} sem nenhuma variável com esse nome.	Criar a variável ou corrigir o nome.
"Rota inexistente" numa rota que existe	Prefixo ou método diferente, ou rota não publicada nesse ambiente.	Comparar com o registro de rotas do ambiente.
Header esperado aparece vazio	Modo navegador sem Access-Control-Expose-Headers.	Expor o header, ou usar o modo Servidor.
Todos os casos autenticados com 401	O token não foi extraído (caminho errado) ou o header <code>Authorization</code> falta no caso.	Conferir a extração do login e os headers.

Dúvidas sobre a ferramenta: fale com o time de QA que compartilhou este manual.